

# Project Title: A Subtitle Describing the Core Contribution

Your Name

23. June 2025

## **Abstract**

This paper presents [Project Name], a [brief, high-level description of the artifact, e.g., a bytecode virtual machine for the Lox language]. The project addresses the problem of [state the core problem] by implementing [describe your core technical approach, referencing key algorithms or architectural patterns]. Key features include [mention 2-3 significant features, e.g., a mark-sweep garbage collector, a Pratt parser, etc.]. The system is evaluated against [mention evaluation criteria, e.g., performance benchmarks, feature completeness against a specification]. The results demonstrate [summarize the main finding, e.g., the viability of a register-based VM for dynamic languages on constrained hardware]. This work serves as a practical application of foundational concepts in [mention the relevant CS field, e.g., programming language implementation].

## 1 Introduction

In the field of [Relevant CS Field], a fundamental challenge is [describe the general problem]. Traditional approaches like [mention a standard approach, e.g., tree-walk interpreters] have known limitations in [mention a key limitation, e.g., performance]. This project, [Project Name], explores an alternative approach by [describe your approach].

The primary goals of this project were to:

- Goal 1: [e.g., Implement a complete bytecode compiler and virtual machine].
- Goal 2: [e.g., Gain practical experience with low-level memory management in C].
- Goal 3: [e.g., Achieve a measurable performance improvement over a prior implementation].

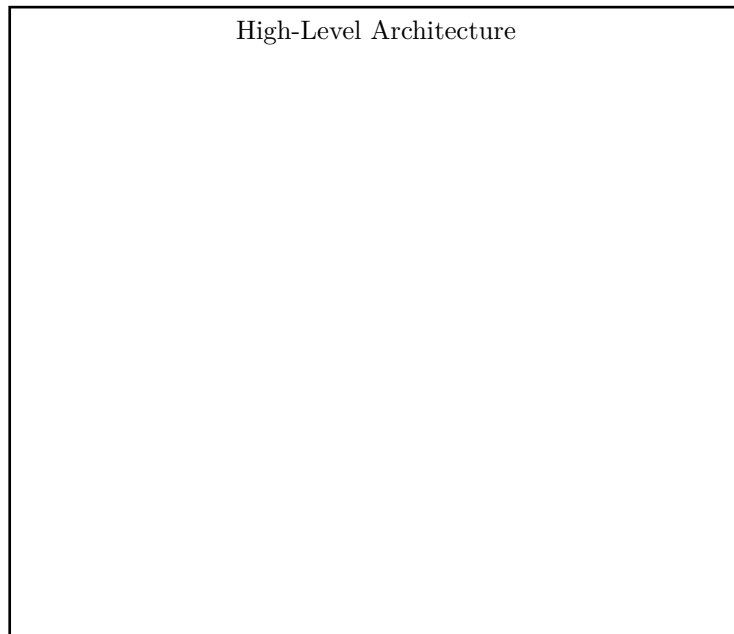
This paper is structured as follows. Section 2 discusses the theoretical foundations. Section 3 details the system's architecture and design. Section 4 covers key implementation challenges. Section 5 presents our performance evaluation. Finally, Section 6 concludes and discusses future work.

## 2 Theoretical Foundations & Related Work

The Raft consensus algorithm, as described by Ongaro and Ousterhout, provides a foundation for building fault-tolerant distributed systems. It decomposes consensus into three subproblems: leader election, log replication, and safety. Our implementation, Raft-FS, models these states directly...

## 3 System Architecture & Design

The system is composed of several major components as illustrated in Figure 1.



**Figure 1:** A diagram of the major components of Project Name.

The choice of [Language/Framework] was motivated by [reason 1, reason 2...]. An alternative, [Alternative Choice], was considered but ultimately rejected due to [explain the trade-off].

### **3.1 Component A: The Parser**

The parser is a top-down recursive descent parser, specifically a Pratt parser, which handles infix expressions with varying precedence...

### **3.2 Component B: The Virtual Machine**

The VM is register-based and features a garbage collector based on the mark-sweep algorithm discussed in [Citation]...

## **4 Implementation Challenges & Solutions**

One of the most significant challenges was the implementation of the garbage collector. Initial attempts led to memory leaks, as seen in the following simplified example:

```

// A snippet of problematic C code.
void someFunction() {
    Object* obj = newObject(TYPE_STRING);
// ... forgot to anchor the object before a potential GC run
collectGarbage(); // Oops! obj might be collected.
}

```

**Listing 1:** Example of an object losing its root before garbage collection.

The solution involved implementing a system of handles to explicitly anchor objects to the stack...

## 5 Evaluation & Results

To evaluate the performance of our bytecode VM, we compared its execution time on a recursive Fibonacci function against the jlox tree-walk interpreter. The benchmark was run on a [Your Machine Specs].

**Table 1:** Execution time for fib(N) in milliseconds.

| Input N | jlox (ms) | clox (ms) |
|---------|-----------|-----------|
| 10      | 50        | 2         |
| 20      | 4800      | 45        |

The results in Table 1 show a performance improvement of approximately two orders of magnitude, which is consistent with the expected overhead of a tree-walk interpreter versus a bytecode VM.

## 6 Conclusion & Future Work

In conclusion, this project successfully demonstrated the implementation of a [artifact] based on principles from [Book/Field]. The primary goals of [state goals] were met, and the resulting system provides a tangible platform for understanding [core concept].

Future work could extend this project in several directions. A just-in-time (JIT) compiler could be added to the VM to further improve performance. Additionally, the type system of the Boba language could be extended to support generics.

## 7 References